ASSIGNMENT – 2

Software Testing and Quality Assurance

MCA – 2nd Year

Detailed answers to Questions 1–15

Prepared for academic study and assignment reference

Note: Where the question does not specify a business rule (for example, grade cut-offs or refund policy), the answer states a reasonable assumption and recommends confirming the official rule.

Q1. Testing strategy for an Online Examination Management System

Goal: verify student registration, subject selection, payments, admit cards and results across the development lifecycle.

Stage/activity	Application	Reason
Static requirements review	Review eligibility, subject rules, fee/refund policy, result access, privacy and admit-card conditions.	Find ambiguity before implementation.
Planning and fundamentals	Define scope, risks, test levels, test data, environments, entry/exit criteria and traceability.	Organises coverage and prioritises high-risk features.
Verification	Review requirements, design, database schema, API contracts and code against specifications.	Checks that work products are built correctly.
Unit testing	Test registration validation, fee calculation, eligibility and result calculations independently.	Finds local logic defects early.
Integration testing	Test registration→payment, payment→admit card, exam submission→result, database and gateway interfaces.	Detects interface and data-consistency defects.
System testing	Run end-to-end eligible/ineligible student, successful/failed payment, download and result-viewing scenarios.	Checks the complete product.
Non-functional testing	Load testing at registration peaks; security, compatibility, usability, accessibility, recovery and performance tests.	Protects records and checks quality attributes.
Validation / acceptance	Students, finance staff and administrators run realistic workflows against acceptance criteria.	Confirms the system meets user needs.
Regression/release	Rerun impacted tests after fixes; smoke-test deployment, backup and rollback.	Prevents regressions and release failures.

Example: a valid student registers, selects an eligible subject, pays once, receives a confirmed payment state, downloads an admit card and later views the authorised result. Also test failed/duplicate payment callbacks and unauthorised result access. Static testing and verification complement dynamic testing; validation confirms fitness for real users.

Q2. Static testing defects in online banking

Static testing reviews requirements, design, code and other artefacts without executing the application. It can detect:

• **Requirements:** unclear transfer limits, missing failure/refund rules, undefined beneficiary verification or bill-payment behaviour.

• **Design:** missing authorisation, weak session management, absent audit logs, unsafe recovery design or no idempotency plan for repeated transfer requests.

• **Code:** incorrect arithmetic, duplicated logic, unreachable branches, null-handling errors, unsafe conversions and inconsistent return values.

• **Security:** hard-coded credentials, secrets in source control, SQL-injection risks, weak cryptography and sensitive account data in logs.

• **Data:** incorrect currency precision, missing database constraints, inconsistent transaction statuses and concurrency risks.

• **Maintainability:** complex functions, poor naming, stale comments and coding-standard violations.

Cost benefit: finding a missing transfer rule during requirements review avoids rework in design, code, tests and deployment. Static analysis also narrows the search area so dynamic tests can focus on runtime, integration and concurrency. Static testing does not replace runtime testing.

Q3. Formal review process for a 2,000-line module

Participants

• Moderator: plans the review and keeps it focused; author: explains and fixes the module; reviewers: inspect logic, style, security and tests; recorder: logs defects; a tester/security specialist may assess testability and risk.

Stages

Stage	Activity
Entry/planning	Confirm module is stable, builds, has requirements/design; define scope, reviewers, checklist and time.
Preparation	Reviewers independently inspect naming, duplication, comments, complex conditions, error handling and tests.

Stage	Activity
Meeting	Author gives context; reviewers cite exact locations and reproducible scenarios; moderator prevents blame and unrelated debates.
Classification	Classify each issue by severity (critical/high/medium/low) and type (correctness/security/performance/maintainability/documentation).
Recording	Log ID, location, description, expected behaviour, severity, category, owner and status.
Rework	Author fixes agreed defects and updates relevant tests without unrelated changes.
Follow-up	Verify fixes, rerun tests, close or formally accept remaining risks, and record lessons learned.

Checklist: meaningful consistent names; focused methods; no unnecessary duplication; simplified conditions; correct boundary/null/error handling; input validation and access control; safe logging; resource cleanup; comments that explain intent; and tests for normal, negative and boundary paths. Exit when findings are recorded and critical/high defects are fixed or formally accepted.

Q4. Peer review versus formal review; e-commerce module procedure

Aspect	Peer review	Formal review
Structure	Often flexible and lightweight.	Defined roles, stages, criteria and records.
Participants	Colleagues review one another's work.	Moderator, author, reviewers and recorder have roles.
Output	Comments, suggestions and defect tickets.	Structured defect log, minutes, metrics and sign-off.
Use	Frequent small changes.	High-risk modules or compliance-sensitive code.

For a five-developer team, choose checkout/payment as the critical module. Developer A authors; B checks business logic; C security; D tests and failure handling; E moderates/records. Review price, tax, discounts, payment idempotency, authorisation, inventory consistency, validation and safe logging.

• Open a pull request with requirements, change summary and test evidence.

• Reviewers independently identify issues and record file/line, scenario, severity and expected correction.

• Discuss disagreements against requirements and reproducible examples; moderator resolves scope issues.

• Author fixes defects and adds/updates tests; reviewers re-check changed lines and rerun relevant tests.

• Merge only after approvals, passing tests and closure or explicit acceptance of high-risk findings.

Peer describes who reviews; formal describes how structured the process is.

Q5. Coding-review checklist: criteria and importance

Criterion	Review rule	Importance
Naming/formatting	Use meaningful names, consistent casing, indentation and braces.	Readable, predictable code.
Function design	Keep methods focused and reasonably small; use single responsibility.	Simpler testing and maintenance.
Duplication/complexity	Avoid copied business rules; simplify nested conditions.	Reduces inconsistent fixes and missed branches.
Types/constants	Use appropriate types; avoid magic numbers and unsafe conversions.	Prevents calculation and conversion errors.
Input validation	Validate format, range, type and length at trust boundaries.	Rejects invalid or malicious data.
Output encoding	Encode untrusted data for the output context.	Reduces injection risks.
Authentication	Use established secure authentication.	Protects accounts.
Authorisation	Check permissions on every protected operation.	Prevents unauthorised access.
Secrets/database	Never hard-code secrets; use parameterised database queries.	Protects credentials and reduces SQL injection.
Error/exception handling	Handle expected failures; avoid swallowed exceptions and internal-detail leaks.	Improves resilience and diagnosis.
Logging	Use suitable levels/correlation IDs; redact sensitive data.	Supports debugging without privacy leaks.
Resources	Close files, streams, sockets and DB connections reliably.	Prevents resource exhaustion.

Criterion	Review rule	Importance
Concurrency	Check shared state, race conditions and transaction boundaries.	Maintains correctness under simultaneous requests.
Performance	Avoid repeated expensive work and unnecessary queries.	Improves response time and scale.
Dependencies/configuration	Use maintained dependencies and safe environment-specific configuration.	Reduces vulnerability and deployment mistakes.
Comments	Explain intent and assumptions; update stale comments.	Keeps documentation trustworthy.
Testing	Cover normal, negative, boundary and regression cases.	Makes behaviour verifiable.
API/accessibility	Follow agreed API contracts and accessibility conventions where applicable.	Improves integration and inclusive use.

Q6. Equivalence Partitioning for Student Result Management

Assumption: each of five subjects accepts integer marks 0–100 inclusive. Total maximum = 500 and percentage = total/5. For illustration, assume a subject pass mark of 40; official grade cut-offs and pass rules must replace this assumption if different.

Partition	Representative data	Expected result
Valid typical	50,60,70,80,90	Accept; total 350, percentage 70%; grade per official scale.
Valid lower boundary	0,40,40,40,40	Accept; total 160, percentage 32%; pass if 40 is inclusive.
Valid upper boundary	100,100,100,100,100	Accept; total 500, percentage 100%.
Below range	-1,50,50,50,50	Reject invalid mark; no valid result.
Above range	101,50,50,50,50	Reject invalid mark.
Non-numeric	abc,50,50,50,50	Field-level validation error.
Decimal if integers required	49.5,50,50,50,50	Reject or follow explicit rounding rule.
Missing mark	blank,50,50,50,50	Require the missing value.
Failing subject	39,80,80,80,80	Marks accepted but overall status Fail under stated assumption.

Equivalence Partitioning chooses a representative from each class expected to behave similarly. Add boundary tests -1, 0, 1, 99, 100 and 101 for each mark field and threshold values 39, 40 and 41. Check that total is at most 500 and percentage stays within 0–100.

Q7. Discount testing using EP and BVA

Purchase amount	Expected discount
Below ■1,000	0%
■1,000–■4,999	5%
■5,000–■9,999	10%
■10,000–■49,999	15%
■50,000 or above	20%

Test amount	Expected result
■999 / ■1,000 / ■1,001	0% / 5% / 5%
■4,999 / ■5,000 / ■5,001	5% / 10% / 10%
■9,999 / ■10,000 / ■10,001	10% / 15% / 15%
■49,999 / ■50,000 / ■50,001	15% / 20% / 20%
■0	0%; verify minimum order rules.
-■1	Reject invalid negative amount.
Blank/text	Validation error.

Use representative class values such as ■500, ■2,000, ■7,000, ■20,000 and ■60,000. Boundary tests reveal off-by-one comparisons, gaps, overlapping rules, wrong percentages, incorrect rule order and rounding errors. Verify both discount and payable amount using the specified currency-rounding policy.

Q8. Test to Pass and Test to Fail for ATM withdrawal

Test to Pass checks valid inputs and normal behaviour. **Test to Fail** supplies invalid or failure conditions and verifies safe rejection/recovery.

Scenario	Expected behaviour
Valid card, correct PIN, active account, sufficient funds and valid amount	Dispense cash once; debit correctly; update history and receipt.
Exact configured limit	Accept or reject according to whether limit is inclusive; rule must be consistent.
Wrong PIN	Reject; count attempts; do not reveal PIN.
Blocked/expired card	Reject; no debit.
Zero, negative or unsupported denomination	Reject; no cash/debit.
Insufficient balance / daily limit exceeded	Reject; balance unchanged.
ATM lacks cash	Do not debit unless dispense is confirmed; reconcile uncertain status.
Network failure before authorisation	No dispense; clear failure/pending state.
Timeout after request	Retry safely; no duplicate debit; reconcile transaction.
Repeated button press / duplicate message	At most one withdrawal per transaction ID.

For each scenario check messages, account balance, dispense status, transaction log, audit trail and recovery. Timeouts and network failures require idempotency and reconciliation because the client may not know whether the server completed the transaction.

Q9. Data testing for hospital appointment system

Data category	Example and expected check
Valid	Valid age, registered doctor, future available slot and successful payment; booking is saved once.
Invalid/boundary	Negative age, past date, age 0/1/17/18 or configured maximum; enforce exact documented limits.
Missing	Age, doctor, date or payment details absent; show required-field error and avoid partial booking.
Wrong type/format	Age='thirty', malformed date or payment reference; reject safely.
Duplicate	Double-click, retry after slow response, or same slot submitted twice; prevent duplicate booking/charge.
Inconsistent	Date of birth conflicts with age; payment says success but booking says unpaid; enforce consistent source of truth.
Extreme	Very long name/notes, huge numbers or large payload; bound input and avoid overflow.
Malicious/unexpected	SQL metacharacters, HTML/JavaScript, unexpected Unicode or malformed JSON; validate, encode and parameterise.
Concurrent	Two patients request last slot; only one booking wins.
Privacy	Patient details in logs, URLs or another user's response; prevent sensitive data exposure.

Use synthetic test data rather than real patient records. Verify UI, API and database behaviour, and ensure failed bookings do not leave partial records or duplicate charges.

Q10. State Transition Testing for bank account

States: New (N), Active (A), Suspended (S), Closed (C). A reasonable model is N→A; A→S on suspicious activity; S→A after authorised review; A→C after closure conditions are satisfied. Confirm exact closure rules with the business owner.

Current state	Event	Expected transition
N	Setup/identity checks complete	N→A (valid)
N	Attempt withdrawal before activation	Remain N; reject operation
A	Suspicious activity detected	A→S (valid)
S	Investigation clears account	S→A (valid)

Current state	Event	Expected transition
S	Attempt restricted transfer	Remain S; reject/restrict
A	Closure conditions satisfied	A→C (valid)
A	Closure conditions unmet	Remain A; reject/defer
C	Attempt transaction/reactivation	Remain C; reject if permanently closed
S	Close account	C only if policy allows; otherwise remain S

Test account creation and activation, suspension, prohibited operations while suspended, authorised reactivation, closure with and without prerequisites, and operations after closure. Include duplicate events and verify audit logs, notifications and balances. Cover every valid transition and important pairs such as A→S→A and A→C.

Q11. State-based testing for food delivery

Current state	Event	Expected next state / behaviour
Order Placed	Payment succeeds	Payment Confirmed
Order Placed	Payment fails	Payment Failed; no restaurant confirmation
Payment Failed	Retry succeeds	Payment Confirmed once
Payment Confirmed	Restaurant accepts	Restaurant Accepted
Payment Confirmed	Restaurant rejects / timeout	Failure or Cancelled path; refund if charged
Restaurant Accepted	Preparation begins	Food Preparing
Food Preparing	Courier picks up ready food	Out for Delivery
Out for Delivery	Delivery proof accepted	Delivered
Eligible pre-cutoff state	Customer cancels	Cancelled; refund according to policy
Charged order cancelled	Refund request accepted	Refund Initiated and tracked
Cancelled/Delivered	Duplicate or late event	Ignore/reject; do not regress state

Test valid transitions and invalid shortcuts such as Order Placed→Delivered. Repeat payment, acceptance, pickup and delivery events; simulate network loss, app restart, courier unavailability and restaurant rejection. Recovery should reconcile uncertain payments, retry with idempotency keys, avoid duplicate notifications/charges, and preserve event logs. Define cancellation/refund eligibility explicitly.

Q12. Decision Table Testing for insurance claims

Illustrative assumption: approve only when policy is valid, premium paid, accident covered, documents complete and amount within policy limit. Amount above limit is referred for manual review in this example.

Condition / action	R1	R2	R3	R4	R5	R6
Policy valid?	Y	N	Y	Y	Y	Y
Premium paid?	Y	-	N	Y	Y	Y
Accident covered?	Y	-	-	N	Y	Y
Documents complete?	Y	-	-	-	N	Y
Amount within limit?	Y	-	-	-	-	N
Action	Approve	Reject	Reject/hold	Reject	Request docs	Manual review

Y=yes, N=no, and '-' means the condition does not change the action already determined. Test one case for each rule: valid claim, invalid policy, unpaid premium, excluded accident, missing documents and over-limit claim. Add exact-limit, zero/negative amount, conflicting records and service failures. Replace illustrative actions with the actual policy before acceptance testing.

Q13. Decision Table Testing for login system

Login succeeds only if username is registered, password correct, account active, account not locked and CAPTCHA valid. There are 2■=32 binary combinations. A compact minimum set groups combinations by the first failed condition, assuming

checks occur in this order.

Rule	U	P	Active	Not locked	CAPTCHA	Expected outcome
R1	Y	Y	Y	Y	Y	Success
R2	N	-	-	-	-	Reject
R3	Y	N	-	-	-	Reject
R4	Y	Y	N	-	-	Reject inactive account
R5	Y	Y	Y	N	-	Reject locked account
R6	Y	Y	Y	Y	N	Reject CAPTCHA

Minimum effective set: R1 plus one test for each failure condition (R2–R6). Add combinations of multiple failures to find order-dependent bugs, and verify that failed logins create no session, rate limits/lockout work, audit events are recorded and messages do not leak sensitive information. If exhaustive combination coverage is required, test all 32 combinations.

Q14. Black-box password test suite

Rules: length 8–20 inclusive, at least one uppercase, lowercase, digit and special character. Confirm the allowed special-character set and Unicode policy in requirements.

Test	Example / condition	Expected result
Valid minimum	Abcdef1! (8 chars)	Accept.
Below minimum	Valid-format 7-character password	Reject.
Minimum boundary	Valid-format exactly 8 characters	Accept.
Maximum boundary	Valid-format exactly 20 characters	Accept.
Above maximum	Valid-format 21 characters	Reject; no silent truncation.
No uppercase	abcdefg1!	Reject.
No lowercase	ABCDEFG1!	Reject.
No digit	Abcdefgh!	Reject.
No special	Abcdefg12	Reject.
Empty/missing	Empty value	Reject as required.
Whitespace	Spaces only or leading/trailing spaces	Apply documented rule; spaces-only fails class requirements.
Unicode/unexpected input	Unicode symbols or quotes/markup	Accept only per documented character policy; never execute as code.
Confirmation mismatch	Password differs from confirmation	Reject if confirmation is required.

Equivalence Partitioning covers valid and invalid character-class groups. Boundary Value Analysis uses lengths 7, 8, 9, 19, 20 and 21. Negative tests cover missing and malformed inputs. Also check HTTPS, salted password hashing, no password in logs and safe error messages; these are additional security checks.

Q15. Static black-box testing of shopping requirements

Static black-box testing reviews requirements, use cases and acceptance criteria without running the application or inspecting internal code. It finds requirement defects before implementation.

Illustrative requirement	Issue	Improvement
“Website loads quickly.”	Ambiguous/unverifiable	Set a measurable target, e.g. 95% of product pages load within 2 seconds under stated load.
“Pay using supported methods.”	Incomplete	List methods, currencies, failures, retries and confirmation behaviour.
“Cancel any time” and “not after dispatch.”	Contradictory	Define the exact state/time after which cancellation is prohibited.
“System is secure.”	Unverifiable	Specify authentication, authorisation, encryption, session timeout and measurable controls.
No out-of-stock rule	Missing	Define block, backorder or substitution behaviour.

Illustrative requirement	Issue	Improvement
“Discount applies to eligible orders.”	Ambiguous	Define eligible products, stacking, threshold, tax/shipping treatment and rounding.
No refund requirement	Missing	Define refund triggers, timing, status and notifications.
Order history without access rules	Incomplete/security gap	Specify that users can view only their own orders and define admin access.

Pre-implementation review checklist

• Are requirements clear, unique, necessary and mutually consistent?

• Are roles, permissions, preconditions and error paths specified?

• Are price, tax, discount, shipping, payment, cancellation and refund rules explicit?

• Are fields, formats, ranges, required values and validation messages defined?

• Are APIs, external services, timeouts, retries and duplicate requests covered?

• Are security, privacy, accessibility, compatibility, retention and audit needs specified?

• Are performance/availability targets measurable with workload and environment stated?

• Can each requirement be mapped to test cases and observable pass/fail outcomes?

• Are boundary, negative, concurrency and recovery cases covered?

• Are assumptions, dependencies, unresolved questions, owners and approvals recorded?

Output a review log containing issue ID, requirement location, category, severity, proposed clarification, owner and status. Ask stakeholders to resolve missing rules rather than inventing acceptance criteria.